Sprint 3 Deliverable

University Inventory Management System

Srinidh & Jithender

Sprint Number:	Sprint 3
Sprint Duration:	2 Weeks (Week 5 – Week 6)
Sprint Goal:	Implement the Inventory Management module (add/edit/delete items) and the QR Code Generation module that auto-assigns a unique QR code to every new item.
Team:	Srinidh, Jithender
Course:	Software Engineering Lab
Date:	April 2026

1. Sprint Plan & Backlog

1.1 Sprint Goal

Enable Admins to perform full CRUD on inventory items. On every new item creation, the system must automatically generate a unique QR code using the qrcode npm library, store its data in the QR_CODE table, and present a printable QR label. This sprint covers UC-004 (Add Item), UC-005 (Edit/Delete Item), and UC-006 (Generate QR Code).

1.2 Sprint Backlog

The following user stories and tasks are selected from the product backlog for Sprint 3.

ID	User Story / Task	Priority	Est. Hrs	Status
US-10	As an Admin, I want to add an inventory item (name, type, room, purchase date) so it is tracked.	High	4	To Do
US-11	As an Admin, I want a QR code to be generated automatically when I add an item.	High	5	To Do
US-12	As an Admin, I want to download or print the QR label for physical attachment to the item.	High	3	To Do
US-13	As an Admin, I want to edit an item's details including room reassignment and status.	Medium	3	To Do
US-14	As an Admin, I want to delete an item and its associated QR code atomically.	Medium	2	To Do
US-15	As an Admin, I want to search and filter items by room, type, or status.	Medium	3	To Do
T-13	Integrate the qrcode npm library into the Node.js backend.	High	3	To Do
T-14	Implement /api/items CRUD endpoints with database transaction handling.	High	5	To Do
T-15	Build Item List page with search/filter controls and a paginated table.	Medium	4	To Do

T-16	Build a printable QR label view showing item ID, name, room, and QR image.	Medium	3	To Do
T-17	Write unit and integration tests for item CRUD and QR generation.	Medium	3	To Do

1.3 Sprint Timeline

Day	Activity
Day 1–2	Finalise item schema; integrate qrcode npm package into the backend
Day 3–4	Implement POST /api/items with auto QR generation inside a DB transaction
Day 5–6	Implement GET (list + filter), PUT, DELETE /api/items endpoints
Day 7–8	Build Add Item form UI; display generated QR image inline after save
Day 9–10	Build Item List page with room/type/status filters and pagination
Day 11	Build printable QR label page (browser print / PDF download)
Day 12–13	Integration tests; fix edge cases (duplicate item IDs, transaction rollback)
Day 14	Bug fixes, sprint review and retrospective

1.4 Definition of Done

- Admin can add an item; a QR code is generated and stored in QR_CODE table in the same transaction.
- QR code image renders on screen immediately after item creation.
- Admin can download or print a QR label showing item ID, name, room, and QR image.
- Admin can edit item fields including room reassignment and status change.
- Admin can delete an item; the QR_CODE row is also removed (cascade or explicit delete).
- Item list supports filter by room, type, and status; pagination at 20 items per page.
- All unit and integration tests pass.
- Code committed with meaningful messages; no secrets stored in the repository.

1.5 Sprint Risks

Risk	Likelihood	Impact	Mitigation
Transaction rollback if QR_CODE insert fails after ITEM insert.	Medium	High	Wrap both inserts in a single DB transaction with try/catch and rollback.
QR generation fails during a bulk import scenario.	Low	Medium	Batch import deferred to backlog; generate one item at a time for now.
Print layout breaks on different paper sizes.	Low	Low	Test print preview; use CSS @media print with fixed label dimensions.
Item ID collisions with custom date-prefixed IDs.	Low	Medium	Auto-generate item_id using prefix + zero-padded daily sequence from DB.

2. Module Design

2.1 Use Cases Addressed

Use Case	Description
UC-004	Admin adds a new inventory item to a room.
UC-005	Admin edits or deletes an existing inventory item.
UC-006	System automatically generates and stores a QR code for each new item.

2.2 Item ID Generation Logic

Item IDs follow the pattern ITM-YYYYMMDD-NNNN where NNNN is a zero-padded daily sequence. Example: ITM-20260404-0001. The backend queries the MAX item_id for the current date prefix and increments the counter, ensuring uniqueness without a separate sequence table.

2.3 QR Code Generation — Backend Snippet

```
const QRCode = require('qrcode');
async function createItemWithQR(conn, itemData) {
await conn.beginTransaction();
try {
const itemId = await generateItemId(conn);
await conn.query('INSERT INTO ITEM SET ?',
{ ...itemData, item_id: itemId });
const qrData = JSON.stringify({ item_id: itemId, name: itemData.item_name });
const qrImage = await QRCode.toDataURL(qrData); // base64 PNG
await conn.query('INSERT INTO QR_CODE SET ?',
{ item_id: itemId, qr_data: qrImage });
await conn.commit();
return { itemId, qrImage };
} catch (err) {
await conn.rollback();
throw err;
}
}
```

2.4 API Endpoints

Method	Endpoint	Description
GET	/api/items	List items; supports ?room_id=, ?type=, ?status=, ?page=
POST	/api/items	Create item and auto-generate QR code (transactional).
GET	/api/items/:id	Get a single item with its QR data.
PUT	/api/items/:id	Update item fields.
DELETE	/api/items/:id	Delete item and its QR code.
GET	/api/items/:id/qr	Return QR code image (base64 or PNG stream).

3. Unit & Integration Test Summary

Test ID	Description

UT-3-01	POST /items creates a row in both ITEM and QR_CODE within the same transaction.
UT-3-02	POST /items with missing item_name returns 400 Bad Request.
UT-3-03	DELETE /items/:id removes both the ITEM row and the QR_CODE row.
IT-3-01	Full flow: Add item → fetch QR → render QR image in UI (Mocha + Supertest).
IT-3-02	Filter: GET /items?room_id=3&status;=Active returns only matching rows.

4. Sprint Review & Retrospective

4.1 Completed Stories

- US-10 through US-15 — All inventory and QR user stories completed.
- T-13 through T-17 — Integration, UI, print view, and test tasks completed.

4.2 What Went Well

- DB transaction pattern handles QR + item atomicity cleanly.
- Reusing the navigation shell from Sprint 2 saved approximately one day of UI work.

4.3 What to Improve

- QR label print CSS took longer than estimated — add print styles to Definition of Done.
- Create a shared Postman collection early to avoid duplicate API testing effort.